PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN SATYA WACANA

Jl. Dr. O. Notohamidjojo 1-10 Blotongan Salatiga 50715 Jawa Tengah

PORTOFOLIO TUGAS WEB SERVICES
TUGAS METHOD POST DENGAN RESTFUL

Nama : Pradena Tiara Maharani
NIM : 672022028
Kelas : Web Service A
Kode Dosen : 67977

1. Kode Program dan Penjelasan

MODEL
(Silakan tambah tabel sendiri, jika model terdiri lebih dari 1)

Kode Program Diketik rapi dengan font `Courier New, 10pt`	Penjelasan Singkat

1.Model User
package com.mobilebanking.model;
 	
public class ModelUser {
private String idUser;
private String username;
private String nama;
private String email;
private String kataSandi;
private String notelp;
private int nomorRekening;
private int pin;
private double saldo;
 	
public String getIdUser() {
return this.idUser;
}
 	
public void setIdUser(String idUser) {
this.idUser = idUser;
}
 	
public String getUsername() {
	ModelUser adalah sebuah class dalam Java yang berfungsi untuk mewakili data pengguna dalam aplikasi mobile banking. Class ini berisi beberapa atribut penting seperti idUser, username, nama, email, dan notelp yang digunakan untuk menyimpan identitas dan kontak pengguna. Selain itu, terdapat atribut kataSandi dan pin yang berfungsi untuk menyimpan informasi keamanan pengguna. Class ini juga mencakup atribut nomorRekening dan saldo untuk menyimpan data rekening serta jumlah saldo pengguna. Setiap atribut pada ModelUser dilengkapi dengan method getter dan

`return this.username;`
`}`
 	
`public void setUsername(String username) {`
`this.username = username;`
`}`
 	
`public String getNama() {`
`return this.nama;`
`}`
 	
`public void setNama(String nama) {`
`this.nama = nama;`
`}`
 	
`public String getEmail() {`
`return this.email;`
`}`
 	
`public void setEmail(String email) {`
`this.email = email;`
`}`
 	
`public String getKataSandi() {`
`return this.kataSandi;`
`}`
 	
`public void setKataSandi(String kataSandi) {`
`this.kataSandi = kataSandi;`
`}`
 	
`public String getNotelp() {`
`return this.notelp;`
`}`
 	
`public void setNotelp(String notelp) {`
`this.notelp = notelp;`
`}`
 	
`public int getNomorRekening() {`
`return this.nomorRekening;`
`}`
 	
`public void setNomorRekening(int nomorRekening) {`
`this.nomorRekening = nomorRekening;`
`}`
	setter yang memungkinkan akses dan modifikasi nilai.

`    public int getPin() {`
`        return this.pin;`
`    }`
 	
`    public void setPin(int pin) {`
`        this.pin = pin;`
`    }`
 	
`    public double getSaldo() {`
`        return this.saldo;`
`    }`
 	
`    public void setSaldo(double saldo) {`
`        this.saldo = saldo;`
`    }`
 	
`}`	

`package com.mobilebanking.model;`
 	
`public class ModelDaftarAntarRekening {`
`    private String nomorRekening;`
`    private String nama;`
 	
`    public String getNomorRekening() {`
`        return this.nomorRekening;`
`    }`
 	
`    public void setNomorRekening(String nomorRekening) {`
`        this.nomorRekening = nomorRekening;`
`    }`
 	
`    public String getNama() {`
`        return this.nama;`
`    }`
 	
`    public void setNama(String nama) {`
`        this.nama = nama;`
`    }`
`}`
	ModelDaftarAntarRekening adalah sebuah class dalam Java yang didefinisikan di dalam package com.mobilebanking.model. Class ini berfungsi sebagai model data untuk menyimpan informasi rekening dalam proses transfer antar rekening. Atribut yang dimiliki oleh class ini meliputi nomorRekening, yaitu nomor rekening bank bertipe String, dan nama, yaitu nama pemilik rekening yang juga bertipe String. Setiap atribut pada ModelDaftarAntarRekening dilengkapi dengan method getter untuk mengambil nilai serta setter untuk mengubah nilai atribut.

`package com.mobilebanking.model;`
 	
`public class ModelDaftarAntarBank {`
 	
`    private String nomorRekening;`
`    private String namaBank;`
`    private String nama;`
	ModelDaftarAntarBank adalah sebuah class dalam Java yang didefinisikan di dalam package com.mobilebanking.model. Class ini berfungsi sebagai model data untuk menyimpan informasi terkait transfer

`public String getNomorRekening() {`
`  return this.nomorRekening;`
`}`
 	
`public void setNomorRekening(String nomorRekening) {`
`  this.nomorRekening = nomorRekening;`
`}`
 	
`public String getNamaBank() {`
`  return this.namaBank;`
`}`
 	
`public void setNamaBank(String namaBank) {`
`  this.namaBank = namaBank;`
`}`
 	
`public String getNama() {`
`  return this.nama;`
`}`
 	
`public void setNama(String nama) {`
`  this.nama = nama;`
`}`
`}`
	antar bank. Atribut yang dimiliki meliputi nomorRekening, yaitu nomor rekening tujuan bertipe String, namaBank, yaitu nama bank tujuan bertipe String, dan nama, yaitu nama pemilik rekening yang juga bertipe String. Setiap atribut pada ModelDaftarAntarBank dilengkapi dengan method getter untuk mengambil nilai serta setter untuk mengubah nilai atribut.

CONTROLLER
(Silakan tambah tabel sendiri, jika controller terdiri lebih dari 1)

Kode Program Diketik rapi dengan font `Courier New, 10pt` `Khusus Method, Path, MediaType dicetak warna biru`	Penjelasan Singkat

`package com.mobilebanking.controller;`
 	
`import java.util.ArrayList;`
`import java.util.List;`
 	
`import javax.ws.rs.Consumes;`
`import javax.ws.rs.POST;`
`import javax.ws.rs.Path;`
`import javax.ws.rs.Produces;`
`import javax.ws.rs.core.MediaType;`
`import javax.ws.rs.core.Response;`
 	
`import com.mobilebanking.model.ModelUser;`
	Di dalam kelas ControllerUser, tersedia beberapa endpoint: untuk registrasi pengguna baru (/user/register), menambahkan nomor rekening dan PIN (/user/tambahkanRekening), login pengguna (/user/login),

```java
@Path("/user")
public class ControllerUser {

    public static List<ModelUser> dataUser = new ArrayList<>();
    public static List<ModelUser> masukakun = new ArrayList<>();
    public static List<String> riwayatTransaksi = new ArrayList<>(); // Ini akan dipakai cek
history juga

    @POST
    @Path("/register")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response register(ModelUser muUserBaru) {
        if (muUserBaru.getUsername() == null || muUserBaru.getKataSandi() == null ||
            muUserBaru.getEmail() == null || muUserBaru.getNama() == null) {
            return Response.status(Response.Status.BAD_REQUEST)
                           .entity("Semua field harus diisi: username, kata sandi, email,
nama.")
                           .build();
        }

        muUserBaru.setIdUser(membuatOtomatisUserId());
        muUserBaru.setSaldo(500000000.0);
        muUserBaru.setNomorRekening(0);
        dataUser.add(muUserBaru);

        String hasil = "User berhasil didaftarkan:\n" +
                "ID User: " + muUserBaru.getIdUser() +
                "\nUsername: " + muUserBaru.getUsername() +
                "\nEmail: " + muUserBaru.getEmail();

        return Response.status(201).entity(hasil).build();
    }

    private String membuatOtomatisUserId() {
        return "USR" + (int)(Math.random() * 100000);
    }

    private int membuatOtomatisNomorRekening() {
        return (int)(100000000 + Math.random() * 900000000); // random 9 digit
    }

    @POST
    @Path("/tambahkanRekening")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
```

mengecek data pengguna (/user/cekuser), mengecek saldo (/user/cekSaldo), dan melihat riwayat transaksi (/user/cekHistory). Data seperti pengguna, akun yang login, dan riwayat transaksi disimpan sementara dalam list statis di memori. Pada kelas ControllerTransaksi, terdapat endpoint untuk melakukan transfer antar rekening dalam satu bank (/transaksi/transferAntarRekening) dan transfer ke rekening bank lain (/transaksi/transferAntarBank). Riwayat transaksi yang terjadi juga disimpan dalam list statis di memori. Di dalam kelas ControllerDaftarAntarRekening, tersedia satu endpoint (/antarRekening/listrekening) yang digunakan untuk mengambil daftar rekening internal dari DaftarAntarRekeningRepository dan mengembalikannya dalam format JSON. Sedangkan pada kelas ControllerDaftarAnta

```
    public Response tambahNomorRekening(ModelUser mutambahrekening) {
        for (ModelUser user : dataUser) {
            if (user.getIdUser().equals(mutambahrekening.getIdUser())) {

                String buatPin = String.valueOf(mutambahrekening.getPin());
                if (buatPin.length() != 6 || !buatPin.matches("\\d+")) {
                    return Response.status(400).entity("PIN harus 6 digit angka.").build();
                }

                user.setNomorRekening(membuatOtomatisNomorRekening());
                user.setPin(mutambahrekening.getPin());

                String hasil = "Nomor rekening berhasil ditambahkan:\n" +
                        "Nomor Rekening: " + user.getNomorRekening() +
                        "\nPIN Rekening: " + user.getPin();

                return Response.status(200).entity(hasil).build();
            }
        }
        return Response.status(404).entity("User tidak ditemukan.").build();
    }

    @POST
    @Path("/login")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response login(ModelUser muLogin) {
        for (ModelUser user : dataUser) {
            if (user.getUsername().equals(muLogin.getUsername()) &&
                user.getKataSandi().equals(muLogin.getKataSandi())) {
                masukakun.add(user);
                return Response.status(200).entity("Login berhasil untuk user: " +
user.getUsername()).build();
            }
        }
        return Response.status(401).entity("Username atau kata sandi salah.").build();
    }

    @POST
    @Path("/cekuser")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response cekUser(ModelUser muCekUser) {
        boolean sudahLogin = false;
        for (ModelUser loggedIn : masukakun) {
            if (muCekUser.getUsername() != null &&
loggedIn.getUsername().equals(muCekUser.getUsername())) {
```

rBank, terdapat satu endpoint (/antarBank/listbank) untuk mengambil daftar rekening bank lain dari RekeningAntarBankRepository, yang juga dikembalikan dalam format JSON melalui HTTP Response.

```
                sudahLogin = true;
                break;
            }
            if (muCekUser.getIdUser() != null &&
loggedIn.getIdUser().equals(muCekUser.getIdUser())) {
                sudahLogin = true;
                break;
            }
        }

        if (!sudahLogin) {
            return Response.status(403).entity("Kamu harus login dulu sebelum cek
user!").build();
        }

        for (ModelUser user : dataUser) {
            if ((muCekUser.getIdUser() != null &&
muCekUser.getIdUser().equals(user.getIdUser())) ||
                (muCekUser.getUsername() != null &&
muCekUser.getUsername().equals(user.getUsername()))) {

                String hasil = "Data User Ditemukan:\n" +
                        "ID User: " + user.getIdUser() +
                        "\nUsername: " + user.getUsername() +
                        "\nNama: " + user.getNama() +
                        "\nEmail: " + user.getEmail() +
                        "\nNo Telepon: " + user.getNotelp() +
                        "\nNomor Rekening: " + (user.getNomorRekening() == 0 ? "Belum ada" :
user.getNomorRekening()) +
                        "\nSaldo: " + user.getSaldo();

                return Response.status(200).entity(hasil).build();
            }
        }

        return Response.status(404).entity("User tidak ditemukan.").build();
    }

    @POST
    @Path("/cekSaldo")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response cekSaldo(ModelUser mucekSaldo) {
        if (mucekSaldo.getIdUser() == null) {
            return Response.status(400).entity("ID User wajib diisi untuk cek
saldo.").build();
        }
```

```
        for (ModelUser user : dataUser) {
            if (user.getIdUser().equals(mucekSaldo.getIdUser())) {
                String hasil = "Saldo saat ini:\n" +
                                "ID User: " + user.getIdUser() + "\n" +
                                "Saldo: Rp" + user.getSaldo();
                return Response.status(200).entity(hasil).build();
            }
        }

        return Response.status(404).entity("User tidak ditemukan.").build();
    }

    @POST
    @Path("/cekHistory")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response cekRiwayat(ModelUser muCekRiwayat) {
        if (muCekRiwayat.getIdUser() == null) {
            return Response.status(400).entity("ID User harus diisi untuk cek riwayat
transaksi.").build();
        }

        List<String> riwayatUser = new ArrayList<>();
        for (String rekam : riwayatTransaksi) {
            if (rekam.contains("User ID: " + muCekRiwayat.getIdUser())) {
                riwayatUser.add(rekam);
            }
        }

        if (riwayatUser.isEmpty()) {
            return Response.status(404).entity("Belum ada transaksi untuk user ini.").build();
        }

        return Response.status(200).entity(riwayatUser).build();
    }
}
```

```
package com.mobilebanking.controller;

import javax.ws.rs.Consumes;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
```

Di dalam kelas ControllerTransaksi, tersedia dua endpoint POST: untuk melakukan transfer antar rekening dalam satu bank (/transaksi/transfer

```
import com.mobilebanking.model.ModelDaftarAntarBank;
import com.mobilebanking.model.ModelDaftarAntarRekening;
import com.mobilebanking.model.ModelUser;
import com.mobilebanking.repository.DaftarAntarRekeningRepository;
import com.mobilebanking.repository.RekeningAntarBankRepository;

@Path("/transaksi")
public class ControllerTransaksi {

    @POST
    @Path("/transferAntarRekening")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response transferAntarRekening(ModelUser muTransferBank) {
        if (muTransferBank.getIdUser() == null || muTransferBank.getNomorRekening() == 0 ||
            muTransferBank.getSaldo() <= 0 || muTransferBank.getPin() == 0) {
            return Response.status(400).entity("Harap isi idUser (pengirim), nomor rekening
tujuan, saldo transfer, dan PIN.").build();
        }

        ModelUser pengirim = null;
        for (ModelUser user : ControllerUser.dataUser) {
            if (user.getIdUser().equals(muTransferBank.getIdUser())) {
                pengirim = user;
                break;
            }
        }

        if (pengirim == null) {
            return Response.status(404).entity("Pengirim tidak ditemukan.").build();
        }

        if (pengirim.getPin() != muTransferBank.getPin()) {
            return Response.status(401).entity("PIN salah. Transfer gagal.").build();
        }

        boolean rekeningTujuanValid = false;
        for (ModelDaftarAntarRekening rekening : DaftarAntarRekeningRepository.get()) {
            if
(rekening.getNomorRekening().equals(String.valueOf(muTransferBank.getNomorRekening()))) {
                rekeningTujuanValid = true;
                break;
            }
        }

        if (!rekeningTujuanValid) {
```

AntarRekening) dan transfer ke rekening bank lain (/transaksi/transferAntarBank). Pada transfer antar rekening, sistem memeriksa kelengkapan input seperti ID user, nomor rekening tujuan, jumlah saldo, dan PIN, lalu memvalidasi pengguna dan rekening tujuan sebelum memproses pemotongan saldo dan mencatat transaksi. Pada transfer antar bank, alur validasi serupa, hanya saja rekening tujuan diverifikasi dari daftar rekening bank lain. Setiap hasil validasi dan proses dikembalikan dalam bentuk HTTP Response dengan status kode yang sesuai seperti 200, 400, 401, atau 404.

```
            return Response.status(404).entity("Nomor rekening tujuan tidak
ditemukan.").build();
        }

        if (pengirim.getSaldo() < muTransferBank.getSaldo()) {
            return Response.status(400).entity("Saldo pengirim tidak cukup.").build();
        }

        pengirim.setSaldo(pengirim.getSaldo() - muTransferBank.getSaldo());

        String riwayat = "Transfer Antar Rekening | " +
                         "User ID: " + pengirim.getIdUser() +
                         " | Ke Nomor Rekening: " + muTransferBank.getNomorRekening() +
                         " | Jumlah: Rp" + muTransferBank.getSaldo();
        ControllerUser.riwayatTransaksi.add(riwayat);

        String hasil = "Transfer berhasil!\n" +
                         "Dari ID User: " + pengirim.getIdUser() + "\n" +
                         "Ke Nomor Rekening: " + muTransferBank.getNomorRekening() + "\n" +
                         "Jumlah: " + muTransferBank.getSaldo();

        return Response.status(200).entity(hasil).build();
    }

    @POST
    @Path("/transferAntarBank")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response transferAntarBank(ModelUser muTransferRekening) {
        if (muTransferRekening.getIdUser() == null || muTransferRekening.getNomorRekening() ==
0 ||
            muTransferRekening.getSaldo() <= 0 || muTransferRekening.getPin() == 0) {
            return Response.status(400).entity("Harap isi idUser (pengirim), nomor rekening
tujuan, saldo transfer, dan PIN.").build();
        }

        ModelUser pengirim = null;
        for (ModelUser user : ControllerUser.dataUser) {
            if (user.getIdUser().equals(muTransferRekening.getIdUser())) {
                pengirim = user;
                break;
            }
        }

        if (pengirim == null) {
            return Response.status(404).entity("Pengirim tidak ditemukan.").build();
        }
```

```
        if (pengirim.getPin() != muTransferRekening.getPin()) {
            return Response.status(401).entity("PIN salah. Transfer gagal.").build();
        }

        ModelDaftarAntarBank rekeningTujuan = null;
        for (ModelDaftarAntarBank rekening : RekeningAntarBankRepository.get()) {
            if
(rekening.getNomorRekening().equals(String.valueOf(muTransferRekening.getNomorRekening()))) {
                rekeningTujuan = rekening;
                break;
            }
        }

        if (rekeningTujuan == null) {
            return Response.status(404).entity("Nomor rekening tujuan tidak ditemukan di bank
lain.").build();
        }

        if (pengirim.getSaldo() < muTransferRekening.getSaldo()) {
            return Response.status(400).entity("Saldo pengirim tidak cukup untuk
transfer.").build();
        }

        pengirim.setSaldo(pengirim.getSaldo() - muTransferRekening.getSaldo());

        String riwayat = "Transfer Antar Bank | " +
                        "User ID: " + pengirim.getIdUser() +
                        " | Ke Nama: " + rekeningTujuan.getNama() +
                        " | Bank: " + rekeningTujuan.getNamaBank() +
                        " | Nomor Rekening: " + rekeningTujuan.getNomorRekening() +
                        " | Jumlah: Rp" + muTransferRekening.getSaldo();
        ControllerUser.riwayatTransaksi.add(riwayat);

        String hasil = "Transfer antar bank berhasil!\n" +
                        "Dari ID User: " + pengirim.getIdUser() + "\n" +
                        "Ke Nama: " + rekeningTujuan.getNama() + "\n" +
                        "Bank Tujuan: " + rekeningTujuan.getNamaBank() + "\n" +
                        "Nomor Rekening Tujuan: " + rekeningTujuan.getNomorRekening() + "\n" +
                        "Jumlah Transfer: Rp" + muTransferRekening.getSaldo();

        return Response.status(200).entity(hasil).build();
    }
}
```

```
package com.mobilebanking.controller;

import java.util.List;
```

Di dalam kelas ControllerDaftarAntarRekening, tersedia

```
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

import com.mobilebanking.model.ModelDaftarAntarRekening;
import com.mobilebanking.repository.DaftarAntarRekeningRepository;

@Path("/antarRekening")
public class ControllerDaftarAntarRekening {

    @GET
    @Path("/listrekening")
    @Produces(MediaType.APPLICATION_JSON)
    public Response getDaftarAntarRekening() {
        List<ModelDaftarAntarRekening> rekeningInternal = DaftarAntarRekeningRepository.get();
        return Response.status(200).entity(rekeningInternal).build();
    }
}
```

REPOSITORY

```
package com.mobilebanking.repository;

import java.util.ArrayList;
import java.util.List;

import com.mobilebanking.model.ModelDaftarAntarRekening;

public class DaftarAntarRekeningRepository {
    public static List<ModelDaftarAntarRekening> aldar = new ArrayList<>();

    static {
        // Dummy data awal
        ModelDaftarAntarRekening daftarantarrekening1 = new ModelDaftarAntarRekening();
        daftarantarrekening1.setNomorRekening("1234567890");
        daftarantarrekening1.setNama("Yevta Sena Putra");

        ModelDaftarAntarRekening daftarantarrekening2 = new ModelDaftarAntarRekening();
        daftarantarrekening2.setNomorRekening("4321098765");
        daftarantarrekening2.setNama("Pradena Tiara Maharani");

        ModelDaftarAntarRekening daftarantarrekening3 = new ModelDaftarAntarRekening();
        daftarantarrekening3.setNomorRekening("7890654321");
        daftarantarrekening3.setNama("Giovanni Sebastian Ekayuda");
```

satu endpoint GET yaitu /antarRekening/listrekening. Endpoint ini digunakan untuk mengambil seluruh data rekening internal yang tersimpan di DaftarAntarRekeningRepository. Sistem akan mengambil daftar rekening dari repository tersebut dan mengembalikannya dalam format JSON melalui HTTP Response dengan status 200 OK.
Repository:
Untuk menyimpan data antar rekening

```
        ModelDaftarAntarRekening daftarantarrekening4 = new ModelDaftarAntarRekening();
        daftarantarrekening4.setNomorRekening("7890654328");
        daftarantarrekening4.setNama("Arnold Rubben Dimas Hattu");

        ModelDaftarAntarRekening daftarantarrekening5 = new ModelDaftarAntarRekening();
        daftarantarrekening5.setNomorRekening("7890654328");
        daftarantarrekening5.setNama("Sherly Septiani");

        aldar.add(daftarantarrekening1);
        aldar.add(daftarantarrekening2);
        aldar.add(daftarantarrekening3);
        aldar.add(daftarantarrekening4);
        aldar.add(daftarantarrekening5);
    }

    public static List<ModelDaftarAntarRekening> get() {
        return aldar;
    }
}
```

```
package com.mobilebanking.controller;

import java.util.List;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

import com.mobilebanking.model.ModelDaftarAntarBank;
import com.mobilebanking.repository.RekeningAntarBankRepository;

@Path("/antarBank")
public class ControllerDaftarAntarBank {

    @GET
    @Path("/listbank")
    @Produces(MediaType.APPLICATION_JSON)
    public Response getDaftarAntarBank() {
        List<ModelDaftarAntarBank> rekeningAntarBank = RekeningAntarBankRepository.get();
        return Response.status(200).entity(rekeningAntarBank).build();
    }
}
```

REPOSITORY

Di dalam kelas **ControllerDaftarAntarBank,** tersedia satu endpoint GET yaitu /antarBank/listbank. Endpoint ini digunakan untuk mengambil semua data rekening antar bank yang tersimpan di RekeningAntarBankRepository. Data yang diambil akan dikembalikan dalam format JSON melalui HTTP Response dengan status 200 OK.

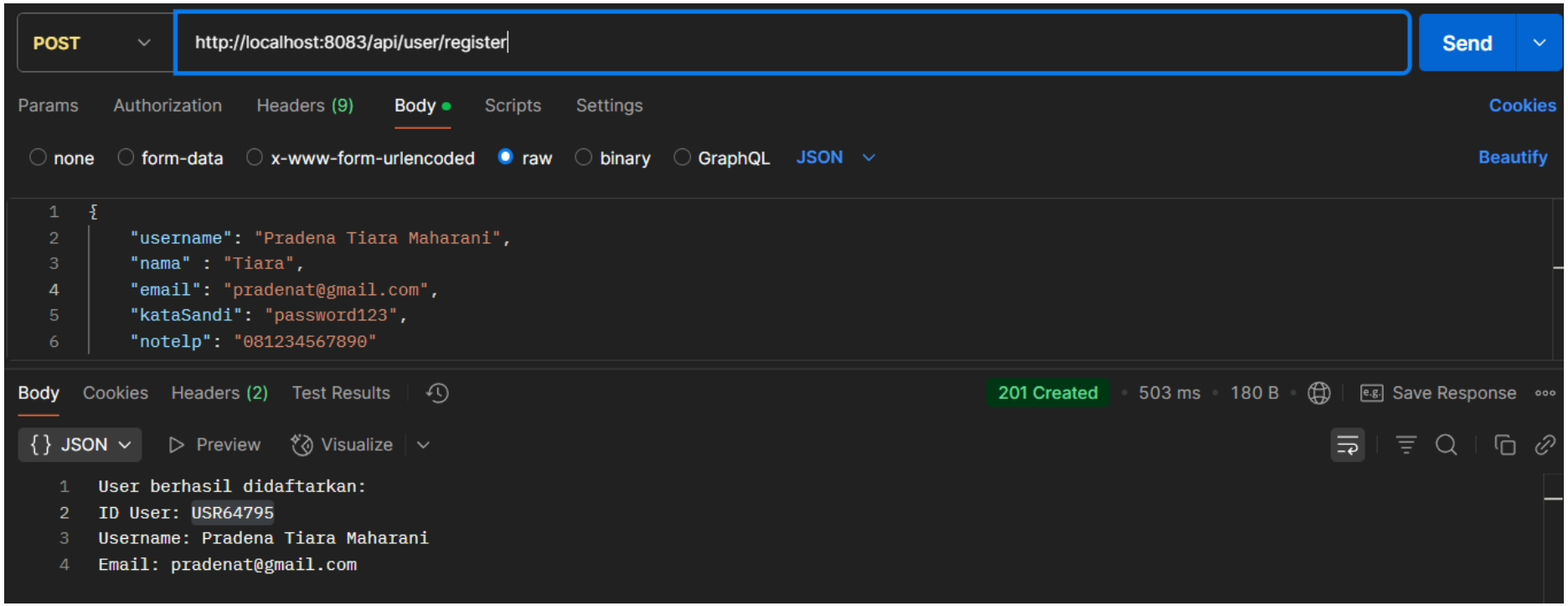
Repository:

```java
package com.mobilebanking.repository;

import java.util.ArrayList;
import java.util.List;

import com.mobilebanking.model.ModelDaftarAntarBank;

public class RekeningAntarBankRepository {
    public static List<ModelDaftarAntarBank> aldab = new ArrayList<>();

    static {
        // Dummy data awal
        ModelDaftarAntarBank daftarantarbank1 = new ModelDaftarAntarBank();
        daftarantarbank1.setNomorRekening("1234567890");
        daftarantarbank1.setNamaBank("BI");
        daftarantarbank1.setNama("Yevta Sena Putra");

        ModelDaftarAntarBank daftarantarbank2 = new ModelDaftarAntarBank();
        daftarantarbank2.setNomorRekening("1321098765");
        daftarantarbank2.setNamaBank("BRI");
        daftarantarbank2.setNama("Pradena Tiara Maharani");

        ModelDaftarAntarBank daftarantarbank3 = new ModelDaftarAntarBank();
        daftarantarbank3.setNomorRekening("1890654321");
        daftarantarbank3.setNamaBank("BRI");
        daftarantarbank3.setNama("Giovanni Sebastian Ekayuda");

        ModelDaftarAntarBank daftarantarbank4 = new ModelDaftarAntarBank();
        daftarantarbank4.setNomorRekening("7890654328");
        daftarantarbank4.setNamaBank("Mandiri");
        daftarantarbank4.setNama("Arnold Rubben Dimas Hattu");

        ModelDaftarAntarBank daftarantarbank5 = new ModelDaftarAntarBank();
        daftarantarbank5.setNomorRekening("7890654328");
        daftarantarbank5.setNamaBank("Mandiri");
        daftarantarbank5.setNama("Sherly Septiani");

        aldab.add(daftarantarbank1);
        aldab.add(daftarantarbank2);
        aldab.add(daftarantarbank3);
        aldab.add(daftarantarbank4);
        aldab.add(daftarantarbank5);
    }

    public static List<ModelDaftarAntarBank> get() {
        return aldab;
    }
```

Untuk menyimpan data antar bank

}	

1. Screenshot 10 Pengujian Service dengan POSTMAN



POST http://localhost:8083/api/user/tambahkanRekening Send

Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2     "idUser" : "USR64795",
3     "pin" : 123456
4 }
```

Body Cookies Headers (2) Test Results 200 OK 17 ms 154 B Save Response

JSON Preview Visualize

```
1 Nomor rekening berhasil ditambahkan:
2 Nomor Rekening: 407630673
3 PIN Rekening: 123456
```

POST http://localhost:8083/api/user/login Send

Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2     "username": "Pradena Tiara Maharani",
3     "kataSandi": "password123"
4 }
```

Body Cookies Headers (2) Test Results 200 OK 21 ms 120 B Save Response

JSON Preview Visualize

```
1 Login berhasil untuk user: Pradena Tiara Maharani
```

POST http://localhost:8083/api/user/cekuser

Send

Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
{
    "username": "Pradena Tiara Maharani",
    "kataSandi": "password123"
}
```

Body Cookies Headers (2) Test Results 200 OK 19 ms 245 B Save Response

JSON Preview Visualize

```
Data User Ditemukan:
ID User: USR64795
Username: Pradena Tiara Maharani
Nama: Tiara
Email: pradenat@gmail.com
No Telepon: 081234567890
Nomor Rekening: 407630673
Saldo: 5.0E8
```

POST http://localhost:8083/api/transaksi/transferAntarRekening Send

Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2   "idUser": "USR64795",      // ID pengirim
3   "nomorRekening": 1234567890,  // Nomor rekening tujuan
4   "saldo": 100000,           // Jumlah uang yang mau dikirim
5   "pin" : 123456
6 }
```

Body Cookies Headers (2) Test Results 200 OK 38 ms 159 B Save Response

{} JSON Preview Visualize

```
1 Transfer berhasil!
2 Dari ID User: USR64795
3 Ke Nomor Rekening: 1234567890
4 Jumlah: 100000.0
```

POST http://localhost:8083/api/transaksi/transferAntarBank Send

Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
{
  "idUser": "USR64795",      // ID pengirim
  "nomorRekening": 1890654321,  // Nomor rekening tujuan
  "saldo": 100000,           // Jumlah uang yang mau dikirim
  "pin" : 123456
}
```

Body Cookies Headers (2) Test Results 200 OK 41 ms 239 B Save Response

JSON Preview Visualize

```
Transfer antar bank berhasil!
Dari ID User: USR64795
Ke Nama: Giovanni Sebastian Ekayuda
Bank Tujuan: BRI
Nomor Rekening Tujuan: 1890654321
Jumlah Transfer: Rp100000.0
```

POST http://localhost:8083/api/user/cekSaldo Send

Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2   "idUser": "USR64795"
3 }
4
```

Body Cookies Headers (2) Test Results 200 OK 23 ms 121 B Save Response

{} JSON Preview Visualize

```
1 Saldo saat ini:
2 ID User: USR64795
3 Saldo: Rp4.998E8
```

POST http://localhost:8083/api/user/cekHistory

Send

Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2   "idUser": "USR64795"
3 }
4
```

Body Cookies Headers (2) Test Results 200 OK 41 ms 314 B Save Response

{} JSON Preview Visualize

```
1 [
2     "Transfer Antar Rekening | User ID: USR64795 | Ke Nomor Rekening: 1234567890 | Jumlah: Rp100000.0",
3     "Transfer Antar Bank | User ID: USR64795 | Ke Nama: Giovanni Sebastian Ekayuda | Bank: BRI | Nomor Rekening: 1890654321 | Jumlah:
        Rp100000.0"
4 ]
```

GET http://localhost:8083/api/antarRekening/listrekening Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (2) Test Results 200 OK 44 ms 382 B Save Response

JSON Preview Visualize


```
[
    {
        "nomorRekening": "1234567890",
        "nama": "Yevta Sena Putra"
    },
    {
        "nomorRekening": "4321098765",
        "nama": "Pradena Tiara Maharani"
    },
    {
        "nomorRekening": "7890654321",
        "nama": "Giovanni Sebastian Ekayuda"
    },
    {
        "nomorRekening": "7890654328",
        "nama": "Arnold Rubben Dimas Hattu"
    },
    {
```

GET http://localhost:8083/api/antarBank/listbank

Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (2) Test Results 200 OK 21 ms 474 B Save Response

JSON Preview Visualize


```
[
    {
        "nomorRekening": "1234567890",
        "namaBank": "BI",
        "nama": "Yevta Sena Putra"
    },
    {
        "nomorRekening": "1321098765",
        "namaBank": "BRI",
        "nama": "Pradena Tiara Maharani"
    },
    {
        "nomorRekening": "1890654321",
        "namaBank": "BRI",
        "nama": "Giovanni Sebastian Ekayuda"
    },
    {
        "nomorRekening": "7890654328",
```